Integrated 3D Perception and Semantic Understanding System on AMtown03

YANG Yuxin 25064265g

CHEN Yushan 25053467g

WANG Shiwei 25042165g

THE HONG KONG POLYTECHNIC UNIVERSITY
香港理工大學

CONTENTS

Part1. Scene Definition & Visual SLAM

Part2. 3D Scene Reconstruction

Part3. Semantic segmentation

Part4. System Evaluation & Global Retrospective

PART 01

Scene Definition & Visual SLAM

Scene Definition

Generalized Urban Sandbox:
AMtown03 is utilized as a high-density urban testbed rather than a single task-specific environment.

Low-Altitude Complexities:
Encapsulates dense geometries, variable textures, and rich semantic elements.

https://mars.hku.hk/dataset.html

Trajectory Tracking & Error Analysis

```
# ORB Extractor: Number of features per image
ORBextractor.nFeatures: 1500

# ORB Extractor: Scale factor between levels in the scale pyramid
ORBextractor.scaleFactor: 1.2

# ORB Extractor: Number of levels in the scale pyramid
ORBextractor.nLevels: 8

# ORB Extractor: Fast threshold
# Image is divided in a grid. At each cell FAST are extracted imposing a minimum response.
# Firstly we impose iniThFAST. If no corners are detected we impose a lower value minThFAST
# You can lower these values if your images have low contrast
ORBextractor.iniThFAST: 20
ORBextractor.minThFAST: 7

#--------------------------------------------------------------------------------------
# Viewer Parameters
#--------------------------------------------------------------------------------------
Viewer.KeyFrameSize: 0.05
Viewer.KeyFrameLineWidth: 1.0
Viewer.GraphLineWidth: 0.9
Viewer.PointSize: 2.0
Viewer.CameraSize: 0.08
Viewer.CameraLineWidth: 3.0
Viewer.ViewpointX: 0.0
Viewer.ViewpointY: -0.7
Viewer.ViewpointZ: -1.8
Viewer.ViewpointF: 500.0
```

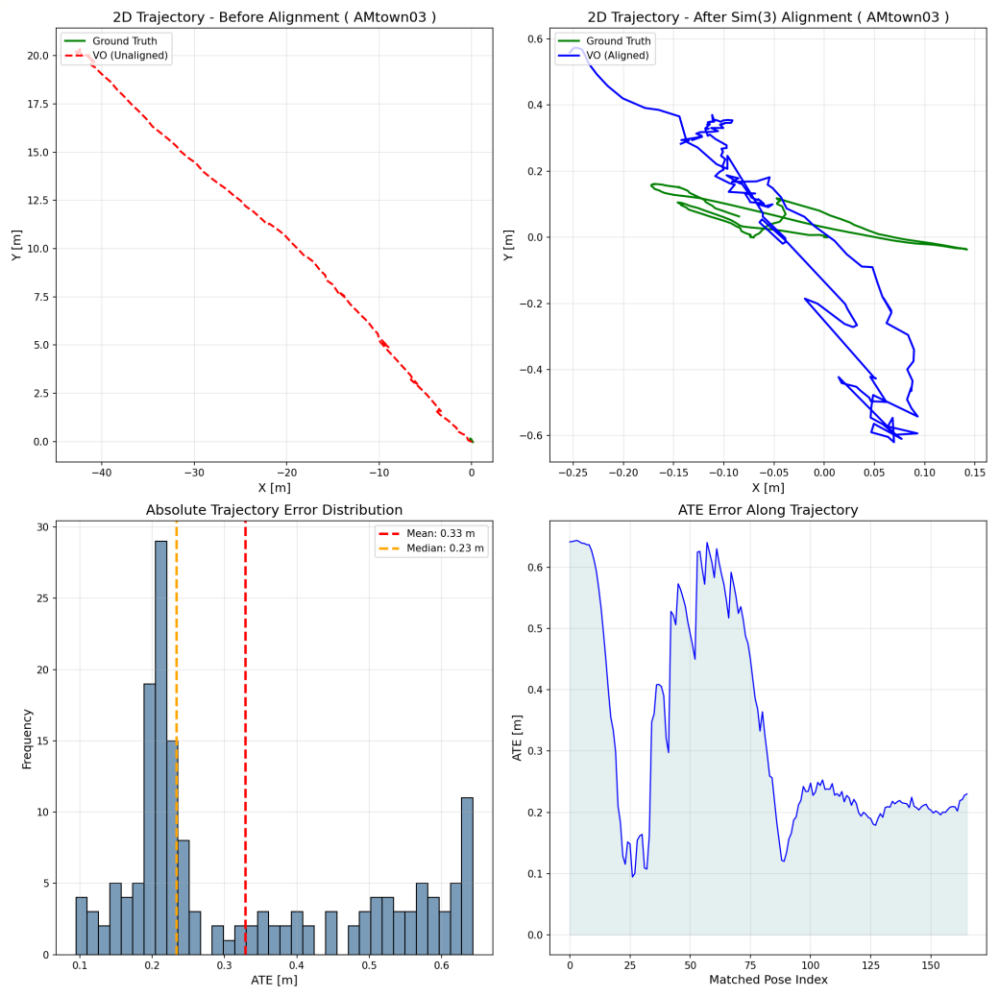
```
# 增加特征点，提升大视角变化与弱纹理场景鲁棒性
ORBextractor.nFeatures: 4200

# 更细的尺度层有助于远近尺度变化
ORBextractor.scaleFactor: 1.15
ORBextractor.nLevels: 12

# 降低 FAST 阈值，弱纹理和暗光场景保留更多角点
ORBextractor.iniThFAST: 10
ORBextractor.minThFAST: 3

#--------------------------------------------------------------------------------------
# 可视化参数
#--------------------------------------------------------------------------------------
Viewer.KeyFrameSize: 0.05
Viewer.KeyFrameLineWidth: 1.0
Viewer.GraphLineWidth: 0.9
Viewer.PointSize: 2.0
Viewer.CameraSize: 0.08
Viewer.CameraLineWidth: 3.0
Viewer.ViewpointX: 0.0
Viewer.ViewpointY: -0.7
Viewer.ViewpointZ: -1.8
Viewer.ViewpointF: 500.0
Viewer.imageViewScale: 1.0
```

Trajectory Tracking & Error Analysis




```
linko@LAPTOP-H83RDI42:~$ python3 AAE5303_group/get_metrics.py
--------------------------------------------
Metric                    | Value
--------------------------------------------
ATE RMSE                  | 0.3702 m
RPE Trans Drift           | 2.2928 m/m
RPE Rot Drift             | nan deg/100m
Completeness              | 15.46 %
--------------------------------------------
```

From a system evaluation perspective, the current algorithm configuration is completely ineffective for the given dataset.

Check rosbag/ IMU

IMU

```
# MARS_LVIG AMtown AMvalley
# Rcl: [0.00298068,-0.999735,-0.0231428,
#       -0.00504595,0.0231320,-0.999740,
#        0.999985,0.00309701,-0.00497598]
# Pcl: [-0.0025563, 0.0567484, -0.0512149]
```

$$T_{bc} = \begin{bmatrix} R_{bc} & t_{bc} \\ 0_{1\times 3} & 1 \end{bmatrix}$$

```
IMU.Tbc: !!opencv-matrix
   rows: 4
   cols: 4
   dt: f
   data: [ 0.00298068, -0.00504595,  0.999985,   -0.0025563,
          -0.999735,    0.0231320,   0.00309701,  0.0567484,
          -0.0231428,  -0.999740,   -0.00497598, -0.0512149,
           0.0,         0.0,         0.0,         1.0]
```

```
# 1. IMU 传感器内参
IMU.NoiseGyro: 0.0017    # 陀螺仪噪声密度 (rad/s/sqrt(Hz))
IMU.NoiseAcc: 0.0200     # 加速度计噪声密度 (m/s^2/sqrt(Hz))
IMU.GyroWalk: 0.0000193 # 陀螺仪偏置随机游走 (rad/s^2/sqrt(Hz))
IMU.AccWalk: 0.0030      # 加速度计偏置随机游走 (m/s^3/sqrt(Hz))
IMU.Frequency: 200       # IMU 频率 (Hz)
```

Spatial Alignment (Extrinsics): Construct the Tbc transformation matrix to physically bind the Camera and IMU coordinate frames.
Noise Modeling: Import hardware calibration parameters to establish algorithmic reliability weights.
Pipeline Switch: Transition the execution pipeline from 'Mono' to 'Mono-Inertial' mode, utilizing high-frequency IMU data to suppress instantaneous visual drift.

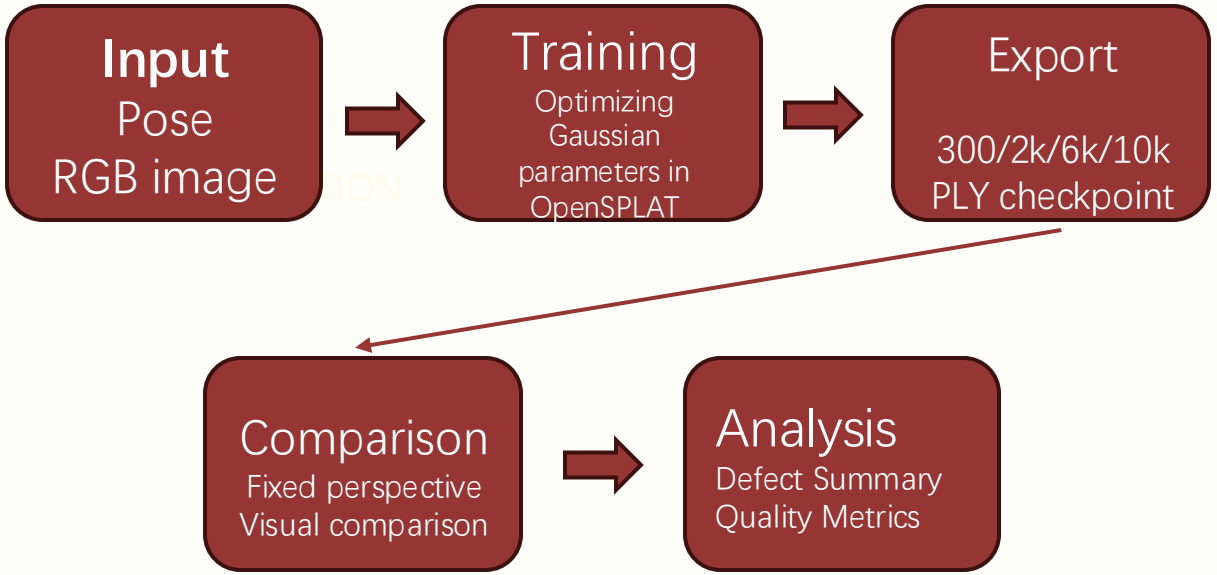
PART 02

3D Scene Reconstruction

Scope of Work and Deliverables

The core value isn't simply "training again," but rather turning the training process into a presentation with clear conclusions:

1) Conduct stage-by-stage comparisons;
2) Explain why the results are good or bad;
3) Select the checkpoint that best represents the final outcome



Why compare multiple checkpoints?

Because in 3DGS training, it's not the case that "the more steps, the more accurate the results"; rather, the goal is to strike a balance between convergence, outliers, and detail preservation.

Final Delivery (10k steps):

- Comparison of reconstruction results across 4 stages
- Defect types: Floating objects / Blurring / Voids
- Proxy quality metrics and best checkpoint

Comparison of Reconstruction Results Across Four Training

300 iterations

The structure is already visible, but it looks dull and sparse overall, and the details haven't been filled in yet.

2000 iterations

The main subject is clearer, but there are still noticeable specks and some blurring in certain areas.

6000 iterations

Coverage continues to improve, though floating objects and stray signals in the outer regions remain noticeable.

10000 iterations

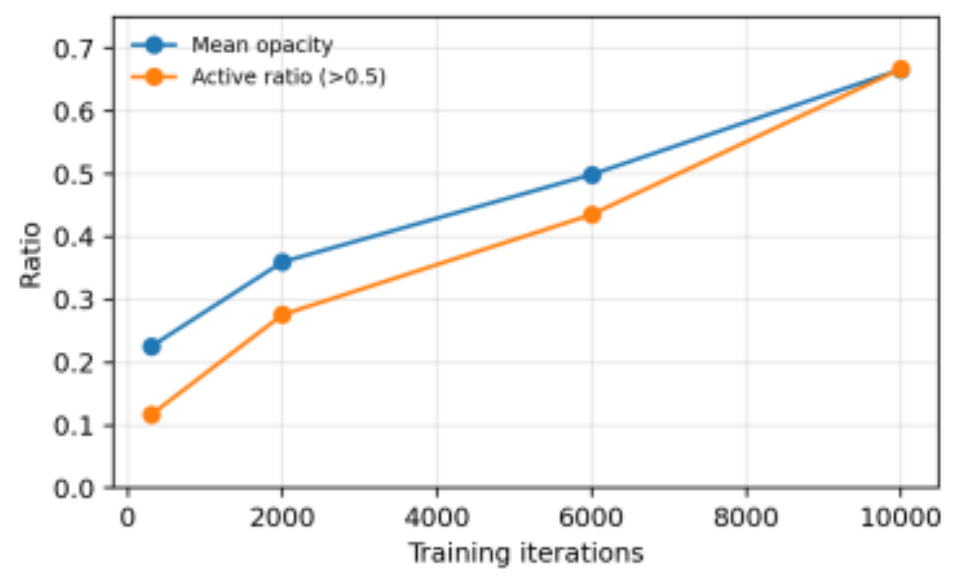
It has the most complete structure and the most stable core, making it the most suitable checkpoint to showcase in this group.

Visually, as the number of training steps increases, the scene gradually evolves from a "rough outline" to "defined main features with added local details." However, the stage prior to 6,000 steps still exhibits noticeable floating elements and blurriness; therefore, it is recommended to prioritize the 10,000-step results for the final presentation.

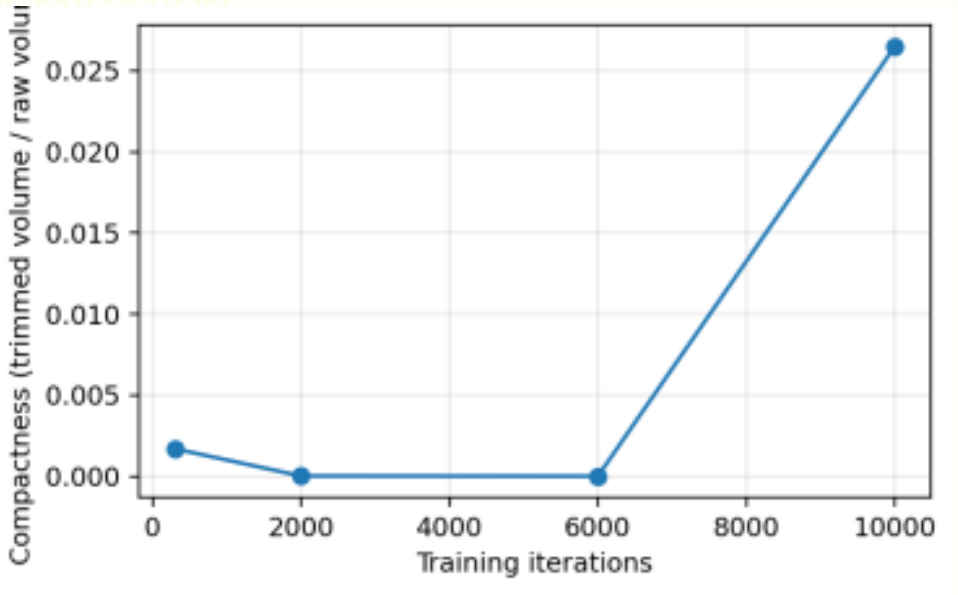
Conclusion:
The 10,000-step checkpoint produces the most complete images, with more natural color contrast and relatively fewer outliers, making it suitable as the primary result to present in the group report.

Reconstructing Quality Indicators: Using “Proxy Indicators” to Assess Convergence Trends

Metric A: Active Ratio



Metric B: raw volume



Explanation:

The steady increase in average opacity and activity ratio indicates that an increasing number of Gaussian components are actively contributing to the imaging process, and the structure of the main body is becoming increasingly clear and stable.

Explanation:

The higher this metric is, the more compact the main object is relative to external outliers. A significant increase of 10,000 steps indicates that the Gaussian blurs far from the main object have converged.

Analysis of Major Defects and Their Causes During the Reconstruction Process

floaters

Symptoms: Discrete patches or scattered dots appear on the exterior of the object.
Stage Assessment:
- 300 steps: A small number of scattered dots are present
- 2000 / 6000 steps: Most pronounced
- 10,000 steps: Some convergence, but not completely gone

Possible Causes: Multi–view geometric constraints are not yet stable enough, causing some Gaussian components to be optimized to incorrect positions.

blur

Observation: Blurred edges; details lack sharpness.
Progress Assessment:
- 300 steps: Most blurry overall
- 2000 steps: Subject is clearer, but still somewhat blurry
- 6000 / 10000 steps: Significant improvement

Possible Cause: During early training, color and scale had not yet fully converged, resulting in a “blurred into a single mass” visual effect after multiple Gaussian filters were applied.

holes

Symptoms: Missing patches; incomplete coverage.
Stage Assessment: • 300 steps: Most noticeable
- 2,000 steps: Insufficient coverage still visible
- 6,000 / 10,000 steps: Most areas have been filled in

Possible Causes: When the field of view is insufficient or the number of training iterations is inadequate, certain areas may not receive sufficient effective Gaussian coverage.

PART 03

SEMANTIC SEGMENTATION

Urban Terrain Semantic Segmentation and Optimization based on U-Net

Background & Objective

With the explosive growth of the Low-Altitude Economy, autonomous navigation and safe obstacle avoidance for UAVs in complex urban environments have become the core pain points for UTM systems. **Semantic segmentation** empowers aerial vehicles with 'pixel-level' environmental perception and an understanding of geometric topology.

01 **Route Planning:** Precisely extracting building boundaries (Roof/Building) to construct safe 3D flight corridors.

02 **Emergency Response:** Real-time identification of paved roads or flat green fields to automatically calculate safe emergency landing zones.

03 **Micro-obstacle Avoidance:** Accurately detecting easily overlooked but fatal micro-obstacles

TARGET

Task Definition: Pixel-level semantic segmentation of drone aerial imagery into 26 distinct categories.

Dataset Overview (AMtown):
Source: AMtown01, 02, 03.
Split Strategy: Train set (01+02)
Test set (03).
Core Challenge: Extreme class imbalance

Baseline Model & Initial Challenges

Pain Point Analysis:

The model fell into a local optimum by overwhelmingly predicting minority classes as "background/green field" (a greedy strategy). Furthermore, the geometric edges of buildings appeared fragmented and distorted.

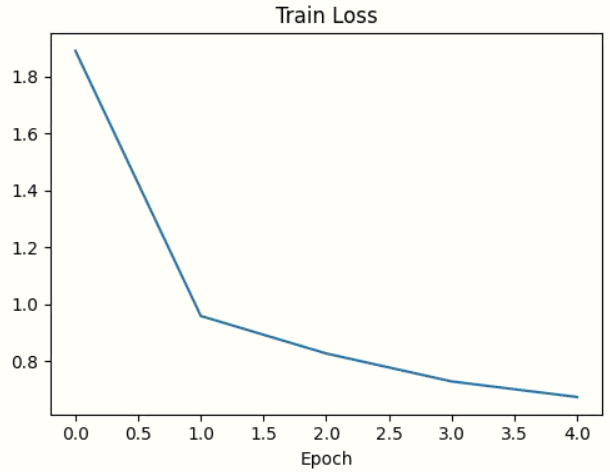
FIRST ATTEMPT

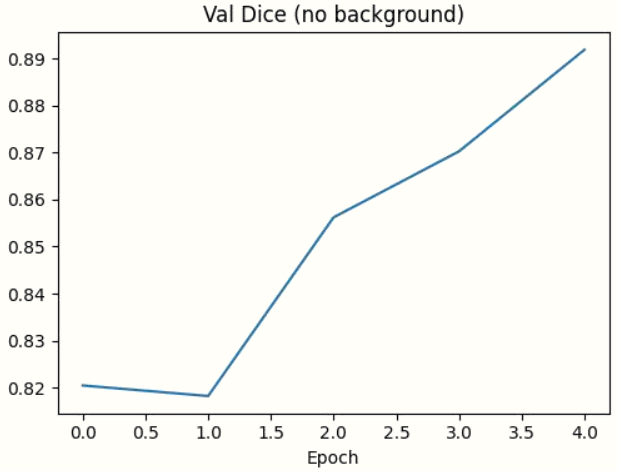
Network Architecture: Classic U-Net architecture (trained from scratch, without pre-trained weights).

Initial Trial & Error (Learning Rate Pitfall):
A standard 1e-4 learning rate caused gradient explosion (NaN).
The optimizer step was precisely recalibrated to a safe 1e-6 to ensure stability.

Baseline Results

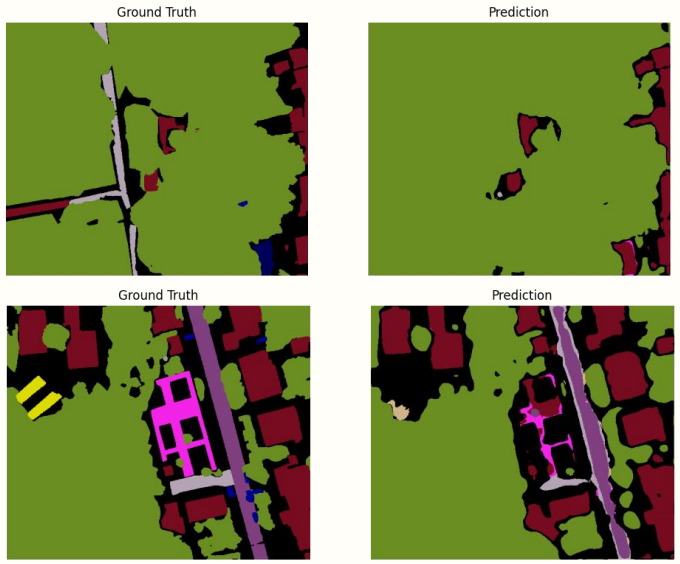
Initial Test Set Performance:
Pixel Accuracy reached 85.5%, but the core evaluation metric Mean IoU was only 24.8%.

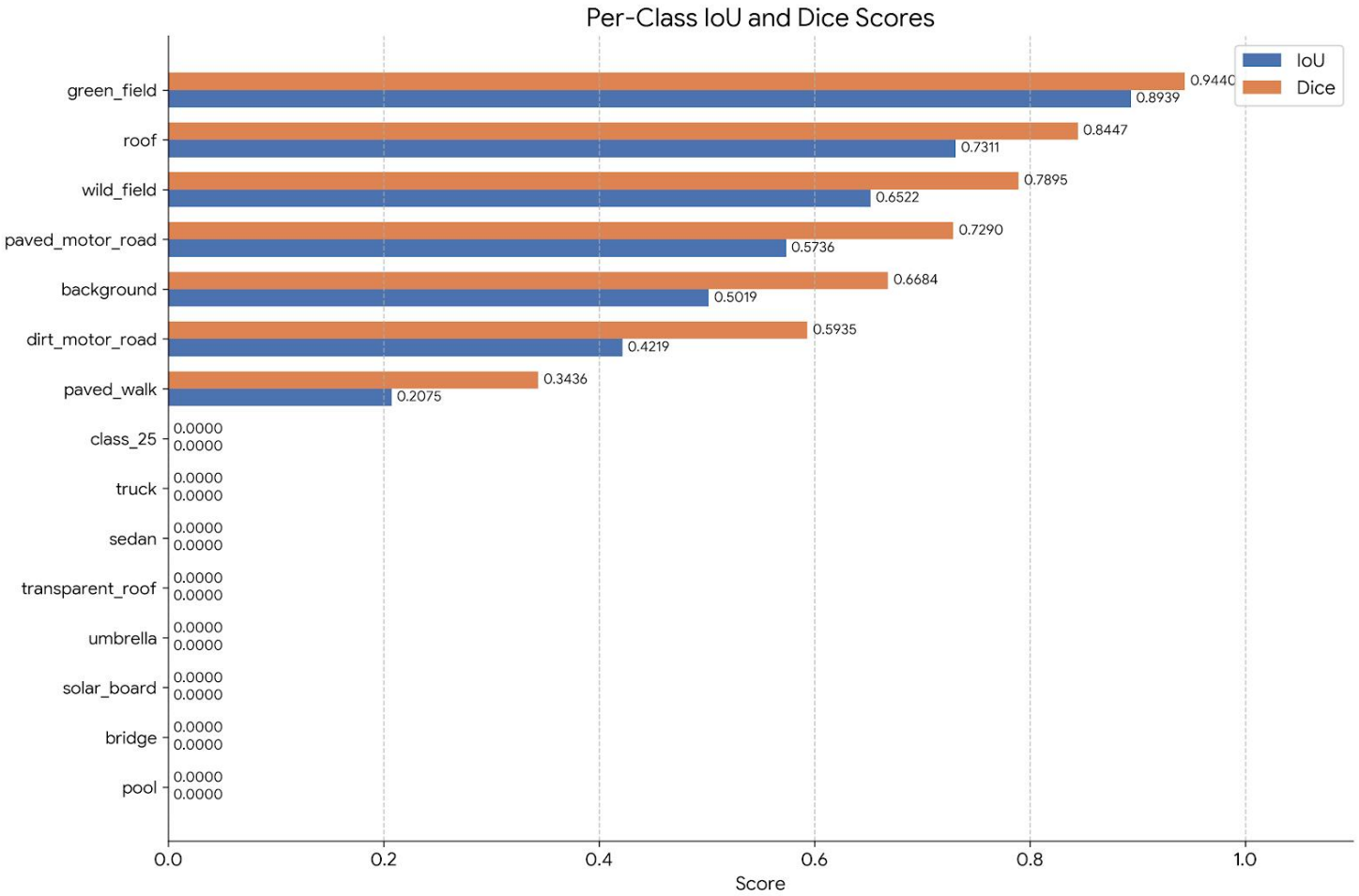




```
Testing:   0%|          | 0/173 [00:00<?, ?img/s]/tmp/ipykernel_3906/3711176180.py:10: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.
  with torch.cuda.amp.autocast(enabled=use_amp):
Testing: 100%|██████████| 173/173 [39:38<00:00, 13.75s/img]
Test metrics (ALL classes)
  pixel_accuracy: 0.855392693384721
  mean_iou      : 0.2654758712607842
  mean_dice     : 0.3275102424528601

Test metrics (IGNORE background class 0)
  pixel_accuracy: 0.855392693384721
  mean_iou      : 0.2485861565002025
  mean_dice     : 0.30316228243709453
```





Optimization Strategy I

Class Weights

Introduced dynamic class weights in the Cross-Entropy loss function to mathematically penalize the long-tail effect.

```
criterion = nn.CrossEntropyLoss(weight=class_weights, ignore_index=255)

history = {'train_loss': [], 'val_dice_no_bg': [], 'lr': []}
import random

for epoch in range(1, epochs + 1):
    model.train()
    epoch_losses = []
    pbar = tqdm(train_loader, desc=f'Epoch {epoch}/{epochs}', unit='batch')

    for batch in pbar:
        images = batch['image'].to(device=device, dtype=torch.float32)
        masks = batch['mask'].to(device=device, dtype=torch.long)
```

Suppressing Dominant Classes

Drastically reduced the loss contribution of background (0.1) and green_field (0.5) to prevent model "laziness".

Boosting Minority Classes

Assigned a 5x weight multiplier to dirt_motor_road and paved_motor_road.

Protecting Micro-Objects

Assigned a heavy 10x weight multiplier to highly vulnerable categories like solar_board and vehicles to force feature extraction.

Optimization Strategy II

Online Data Augmentation

Prevent the neural network from memorizing the absolute pixel coordinates of the training sets , thereby improving generalization on unseen domains .

```
# --- 在线数据增强 ---
if random.random() > 0.5:
    images = torch.flip(images, dims=[3]) # 水平
    masks = torch.flip(masks, dims=[2])
if random.random() > 0.5:
    images = torch.flip(images, dims=[2]) # 垂直
    masks = torch.flip(masks, dims=[1])
```

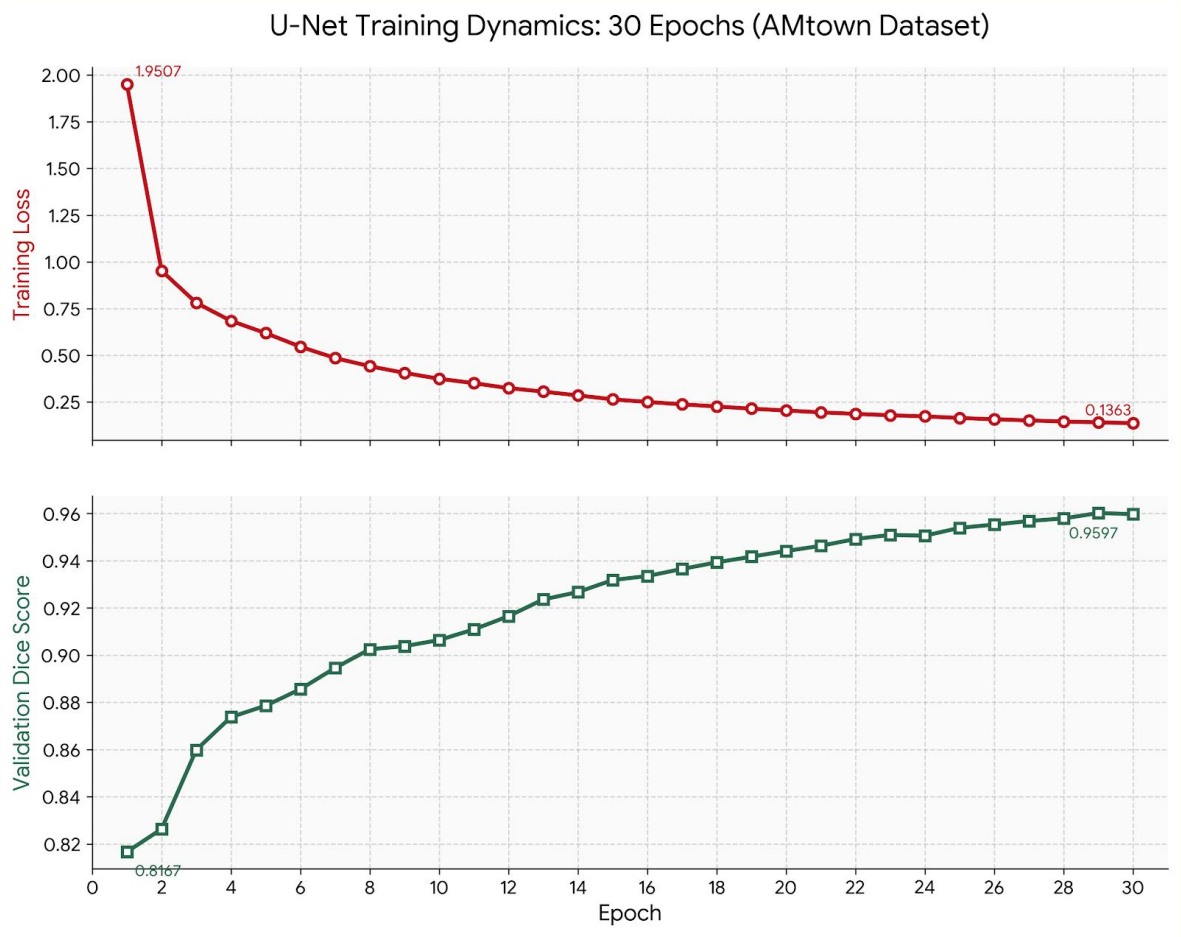
Implementation Details

Applied a 50% probability of synchronous random horizontal and vertical flips dynamically during the DataLoader process.

Underlying Logic

Stripping the model of spatial memorization forces the convolutional kernels to learn the true physical topology, structural boundaries, and texture features of the urban environment.

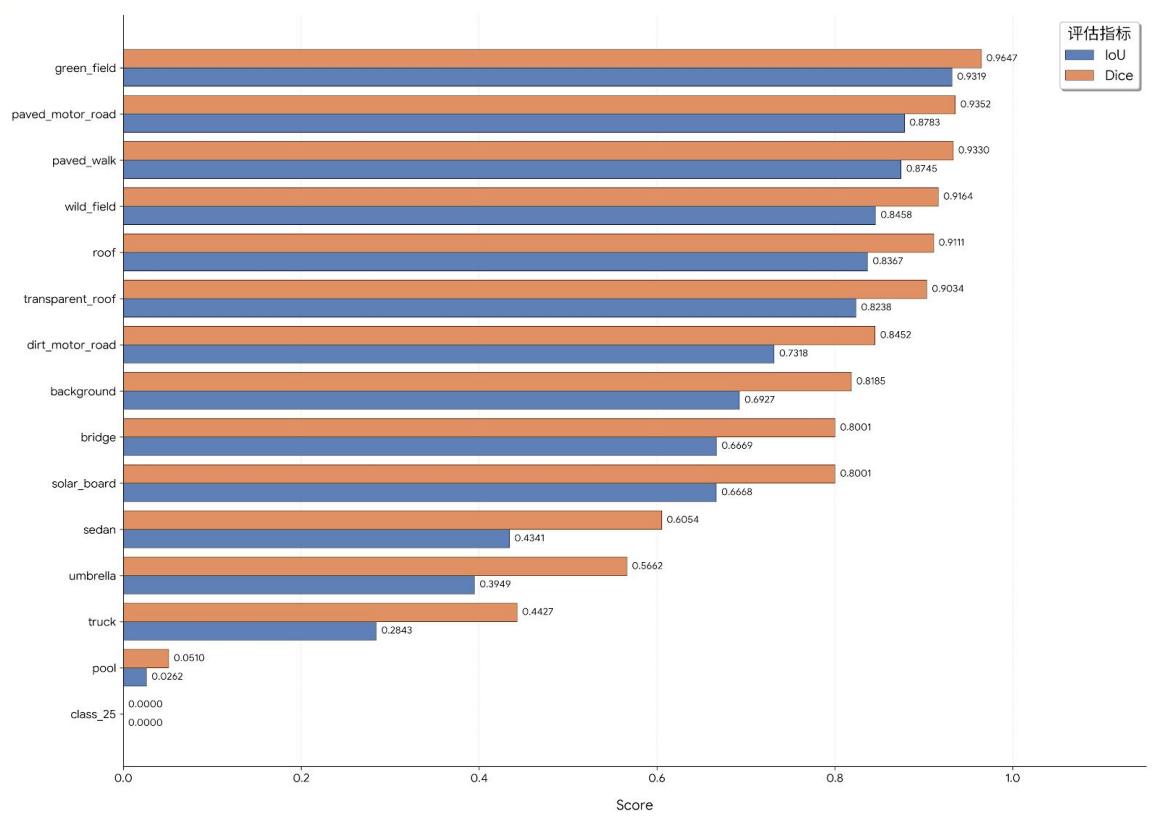
Quantitative Results

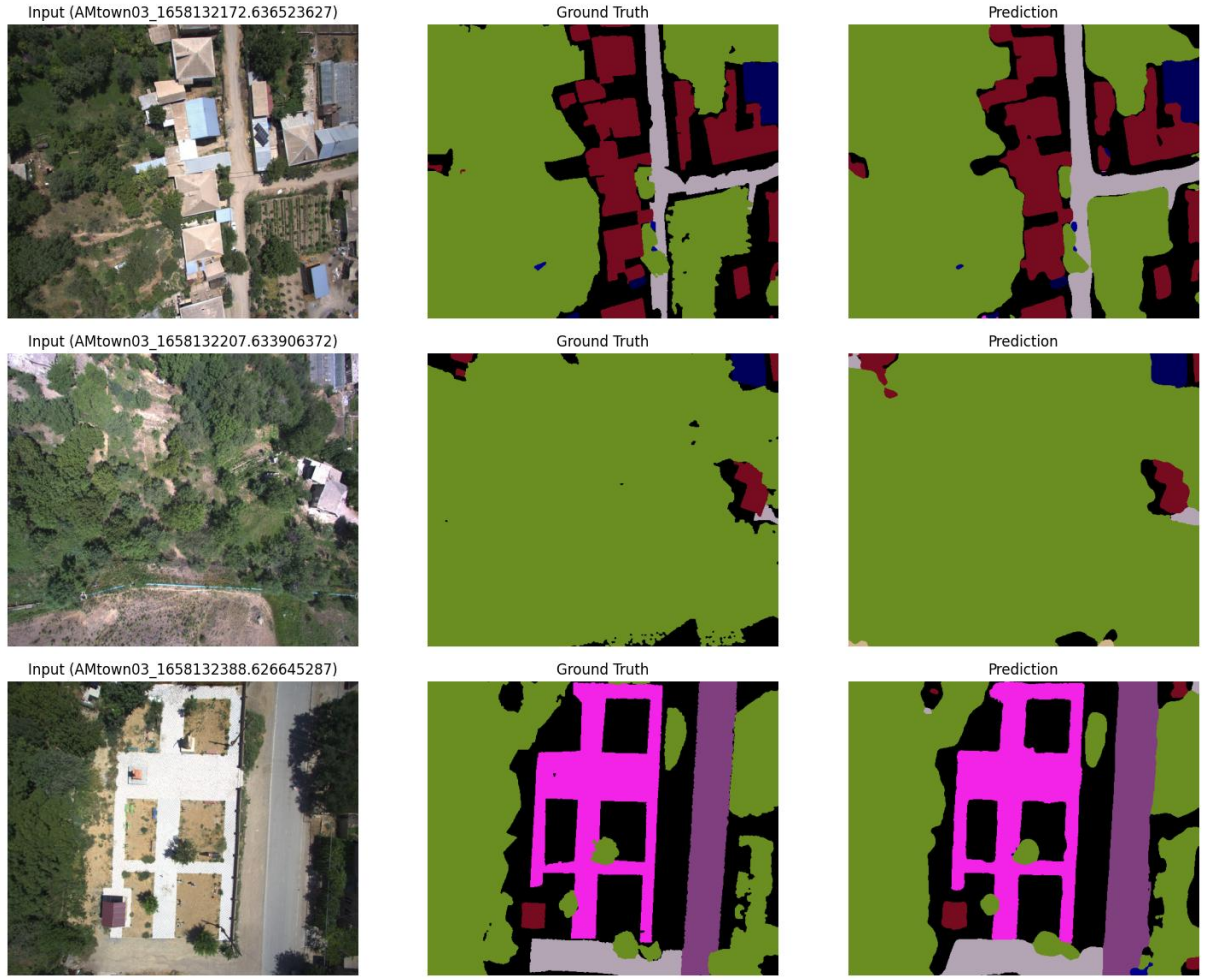



```
Testing: 100%|██████████| 140/140 [07:14<00:00,  3.10s/img]
Test metrics (ALL classes)
  pixel_accuracy: 0.9220516229766647
  mean_iou      : 0.6059087090408587
  mean_dice     : 0.6995402825318087

Test metrics (IGNORE background class 0)
  pixel_accuracy: 0.9220516229766647
  mean_iou      : 0.5997078423819724
  mean_dice     : 0.6910452834384555
```

Quantitative Results





Conclusion & Future Work

Conclusion: Under severe computational constraints (global downscaling SCALE=0.25) and extreme dataset imbalance, the U-Net's baseline potential was maximized to an excellent 60% mIoU through targeted loss penalization and spatial augmentation.

Future Optimization (Limitations & Next Steps)

Modifying the Data Feeding Strategy — From "Global Downscaling" to "Patch-based Training"

Implementation: Discard SCALE = 0.25 and maintain the original high-definition resolution (SCALE = 1.0). To prevent VRAM overflow (CUDA OOM), modify the Dataset loading logic to randomly crop a 512x512 patch from the original image for each training iteration. During inference (testing), utilize a "Sliding Window" technique to seamlessly stitch together the complete prediction map.

Adopting Advanced Loss Functions

Implementation: The current CrossEntropy + Dice combination is solid, but Focal Loss could be introduced. The core philosophy of Focal Loss is "hard sample mining": once the model becomes highly confident in predicting easy classes (like the background), those pixels will no longer contribute significantly to the total loss. This forces the neural network to concentrate its computational capacity on struggling with ambiguous boundaries and visually similar classes.

PART 04

System Evaluation Global Retrospective

A comprehensive overall review of the entire system's performance

System Integration

End-to-End Data Flow
Multimodal Alignment

Integration Philosophy:

End-to-End Pipeline Implementation
Achieve automation and precise alignment of the entire data flow.

Visual Demonstration:

Co-Perspective Alignment of Multimodal Data
Although high-density 3D point cloud rendering and fusion are not currently implemented, the system's foundational layer has already achieved matrix alignment of both data modalities via camera intrinsics.

Future Outlook

Content: A comparison of the 3D geometric structure and the 2D semantic perception results from the same distinct viewpoint.

Technical Explanation: Although high-density 3D point cloud rendering and fusion are not currently implemented, the system's foundational layer has already achieved matrix alignment of both data modalities via camera intrinsics.

Significance: This set of aligned images demonstrates the rudimentary form of our "3D Semantic Cognitive Map."